



CyberSentinel

INSTALLATION & ADMINISTRATION GUIDE · V2.0.0

Install & run CyberSentinel on a server

Two ways to install: **A)** from the GitHub repository (builds on the server), or **B)** from the offline image bundle (no source, for banks). Both end with the dashboard on `http://<server>:19888`. Then configure the AI key, and rotate it any time.

1. Prerequisites — install Docker (one time)

On a fresh Ubuntu/Debian server, install Docker Engine + the Compose plugin:

```
# quickest – official convenience script
curl -fsSL https://get.docker.com | sudo sh

# let your user run docker without sudo, then log out and back in
sudo usermod -aG docker $USER

# verify (need Docker 24+ and the compose v2 plugin)
docker --version
docker compose version
```

RHEL/CentOS: the same convenience script works. Minimum server: 16 GB RAM, 50 GB disk, and port 19888 reachable by analysts.

2. Option A — One-command install (recommended)

The whole product installs from a single master script. Copy `scripts/bootstrap.sh` onto the fresh server (paste it into a file, or `scp` it over) and run it:

```
bash bootstrap.sh
```

It asks for **one thing — your GitHub Personal Access Token** — then does everything: clones the product, writes its configuration (the AI key and store password are already baked in, so you type nothing else), builds and starts all services, applies the database schema, verifies health, and installs the `aiml` operator command. When it finishes it prints the live URL — open `http://<server-ip>:19888`.

That is the entire install. The token is the only input; it is prompted (never stored in the script), used to clone, then stripped from the git config. The script is safe to re-run — an existing install updates in place and existing data is never touched. From then on the stack is run with `aiml start | stop | status | health` (§8).

Prerequisites: Docker, git and curl on the box (`sudo apt-get install -y curl git` + Docker per §1). **Token:** create one at `github.com/settings/tokens` with the `repo` scope.

If you prefer the manual steps

```
# 1 – get the code (token = your PAT when Git prompts)
git clone https://github.com/Tejasvgipl/AIML-V2.git aiml
cd aiml

# 2 – install everything (builds, starts, applies the schema, verifies, bakes the AI key)
bash scripts/install.sh
```

When it finishes it prints the live URL. Open `http://<server-ip>:19888` in a browser.

What `install.sh` does for you. It checks Docker, creates `.env` from the template, builds and starts all six containers, waits for the event store, applies the database schema, and confirms `"store": "connected"` before it reports success. It also **auto-recovers from the two failures that used to break fresh installs** — a leaked-Docker-network "cannot allocate memory" error, and a schema that was skipped on first boot. It is safe to re-run at any time; it never deletes data. Finally it installs the `aiml` operator command, so from then on you run the stack with `aiml start` / `aiml stop` / `aiml health` (see §8).

If you prefer to run the steps yourself

```
cp .env.example .env          # UI on port 19888, AI off by default
./scripts/setup.sh            # optional: AI key + store password
docker compose --profile wazuh up -d --build
curl -s localhost:19888/api/health
# {"status":"ok","store":"connected","ai":"configured"|"missing"}
```

Private repo? When Git asks for a password, paste a GitHub **Personal Access Token** (not your login password), or run `gh auth login` first.

Update later: run `aiml update` (does `git pull` + rebuild for you), or manually `git pull` then `docker compose --profile wazuh up -d --build`. The data volume is preserved.

3. Option B — Install from the offline image bundle (no source)

Use this for a customer/bank server that must not receive the source code. You hand over the `cybersentinel-<version>/` bundle folder (built with `scripts/package.sh`).

- 1 Copy the bundle folder to the server and enter it:

```
cd cybersentinel-2.0.0
```

- 2 Run the one-command installer:

```
./run.sh
```

It loads the images from `images.tar.gz`, runs `setup.sh` for the AI key + store password on first run, and starts the stack.

- 3 Verify & open — same as above: `curl -s localhost:19888/api/health`, then `http://<server>:19888`.

Build the bundle (on your machine, one command): `./scripts/package.sh 2.0.0` → produces `dist/cybersentinel-2.0.0/` with no source inside.

4. Point log ingestion at your alerts

The watcher reads your `alerts.json` **read-only** and streams events into the store. Set the path in `.env`, then restart with the ingestion profile:

```
# in .env
WAZUH_ALERTS_DIR=/var/ossec/logs/alerts      # directory holding alerts.json

# apply
docker compose --profile wazuh up -d          # GitHub install
docker compose -f docker-compose.prod.yml --profile wazuh up -d  # bundle install
```

5. AI analyst & API key

The AI analyst powers SOC Query, alert explanations and incident narratives. It is the **only** feature that can send data outside the server, and **only** when an analyst clicks an AI panel or runs a query — never automatically. Configure it with:

```
./scripts/setup.sh
```

Mode	You enter	Data leaves the server?
1 · Cloud	Your Groq API key	Yes — short log snippets, on-demand only
2 · On-prem LLM	Your OpenAI-compatible URL + key + model	No — stays inside your network
3 · Disabled	nothing	No — zero external calls

Where the key lives: only in the local `.env` file (chmod 600, never in the images, never committed). Each server has its own.

Setting it manually in `.env`

```
# Cloud (Groq):  
GROQ_API_KEY=your-key-here  
  
# On-prem / self-hosted OpenAI-compatible endpoint:  
GROQ_API_KEY=  
AI_BASE_URL=https://llm.internal.bank/v1/chat/completions  
AI_API_KEY=your-endpoint-key  
AI_MODEL=your-model  
AI_FAST_MODEL=your-model  
  
# Disabled: leave GROQ_API_KEY and AI_API_KEY empty
```

6. Change / rotate the API key

No code change and no rebuild — it is runtime config:

- 1 Re-run the configurator (or edit `.env`): `./scripts/setup.sh`
- 2 Recreate the backend so it reads the new value:

```
docker compose up -d backend # GitHub install  
docker compose -f docker-compose.prod.yml up -d backend # bundle install
```

- 3 Confirm: `curl -s localhost:19888/api/health` → `"ai":"configured"`

A plain `docker restart` is not enough — env values are read when the container is *created*, so use `up -d backend` to recreate it.

7. Ports & store password

All overridable in `.env`:

Variable	Default	Purpose
NGINX_HOST_PORT	19888	Dashboard + API (the analyst URL)
BACKEND_HOST_PORT	19110	Backend (direct)
ML_ENGINE_HOST_PORT	19111	ML engine (direct)
CLICKHOUSE_PASS	—	Event-store password (set via setup.sh)

Never run `docker compose ... down -v` — the `-v` deletes the data volume (your entire log store). Plain `down` keeps the data.

8. Everyday operations

After `install.sh` has run once, the whole stack is driven by a single command — `aiml`. It works the same on a GitHub install and a bundle install (it detects which one it is), so operators never need to remember Docker syntax.

Action	Command
Start everything	<code>aiml start</code>
Stop — pause, data kept	<code>aiml stop</code>
Restart everything	<code>aiml restart</code>
Quick status	<code>aiml status</code>
Full health report	<code>aiml health</code>
Follow logs (all / one service)	<code>aiml logs</code> · <code>aiml logs backend</code>
Update to latest code (GitHub install)	<code>aiml update</code>
Version / commit	<code>aiml version</code>

`aiml health` **is the one to run when something looks off.** It reports every container's state, the event store (reachable, password OK, live **row count** and on-disk size), the backend API and store connection, the risk/ML engine, the ingestion watcher, and host disk headroom — ending in a clear **HEALTHY** or **ATTENTION** verdict.

`aiml stop` **is a safe pause** (`docker compose stop`). Containers halt; the event-store volume is never touched. The tool has no data-deleting command at all.

Raw Docker equivalents (if you ever need them)

Action	GitHub install	Bundle install
Start	<code>docker compose --profile wazuh up -d</code>	<code>docker compose -f docker-compose.prod.yml --profile wazuh up -d</code>
Status	<code>docker compose [-f docker-compose.prod.yml] ps</code>	
Logs	<code>docker compose [-f docker-compose.prod.yml] logs -f backend</code>	
Stop (keep data)	<code>docker compose [-f docker-compose.prod.yml] stop</code>	

9. Troubleshooting

Try this first: `bash scripts/install.sh` . It is safe to re-run, and it repairs most of the problems below automatically (schema, startup, health).

The three install failures — and their exact fix

What you see	Why	Exact fix
The UI says “store degraded”, or health returns <pre>"store": "connection_failed"</pre>	The password in <code>.env</code> is not the password the store's data volume was created with, so the services can't authenticate. (Fixed at the source in v2.0.1 — both now come from the same <code>CLICKHOUSE_PASS</code> value.)	<pre>bash scripts/install.sh</pre> It tests the credential and tells you which side is wrong. To change the password later, always use <code>./scripts/setup.sh</code> — it updates the store <i>and</i> <code>.env</code> together.
<pre>Database cybersentinel does not exist</pre>	The schema was skipped during the store's first boot.	Re-apply it (safe on a store that already holds data — every statement is <code>CREATE ... IF NOT EXISTS</code>): <pre>docker exec -i aiml_clickhouse clickhouse-client \ --password "\$CLICKHOUSE_PASS" -- multiquery \ < clickhouse/init/01- schema.sql</pre>
<pre>failed to create endpoint ... cannot allocate memory</pre>	Not a RAM shortage. Docker leaked virtual network interfaces from earlier failed attempts.	<pre>sudo systemctl restart docker docker compose --profile wazuh up -d</pre>
<pre>container aiml_clickhouse is unhealthy</pre> on the very first start	The store takes longer than the wait window to initialise on a cold server.	Simply run the start command again — it succeeds on the second attempt: <pre>docker compose --profile wazuh up -d</pre>

Everything else

Symptom	What to check
Dashboard won't load	<code>docker compose ps</code> — is the backend healthy? <code>curl localhost:19888/api/health</code>
AI panels say "not configured"	No key set — run <code>./scripts/setup.sh</code> , then <code>up -d backend</code>
SOC Query says "service busy"	AI provider rate-limited — retry shortly, or use a paid tier / on-prem model
No events appear	Check <code>WAZUH_ALERTS_DIR</code> and <code>logs -f wazuh-watcher</code>
Private clone fails	Use a GitHub Personal Access Token as the password, or <code>gh auth login</code>

Health one-liner: `curl -s localhost:19888/api/health` should return `"store":"connected"`.